📏 Sorting Height-Anomalous Color Blocks

1. Course Overview

The function implemented in this lesson can be simply understood as: **camera measures height, identifies stacked blocks, prioritizes grasping the taller one**.

Combining color threshold segmentation with 3D spatial height estimation model, the program calculates the top surface spatial height of each colored block on the table in real-time, automatically distinguishing "normal height" (single layer, approximately 3cm) from "anomalous height" (double-layer stacked, approximately 6cm). After pressing spacebar, it prioritizes locating and grasping the height-exceeding anomalous blocks for classified unloading.

> **Note**
>
> **The tutorial recommends using the accompanying 3×3cm colored wooden blocks.**

---

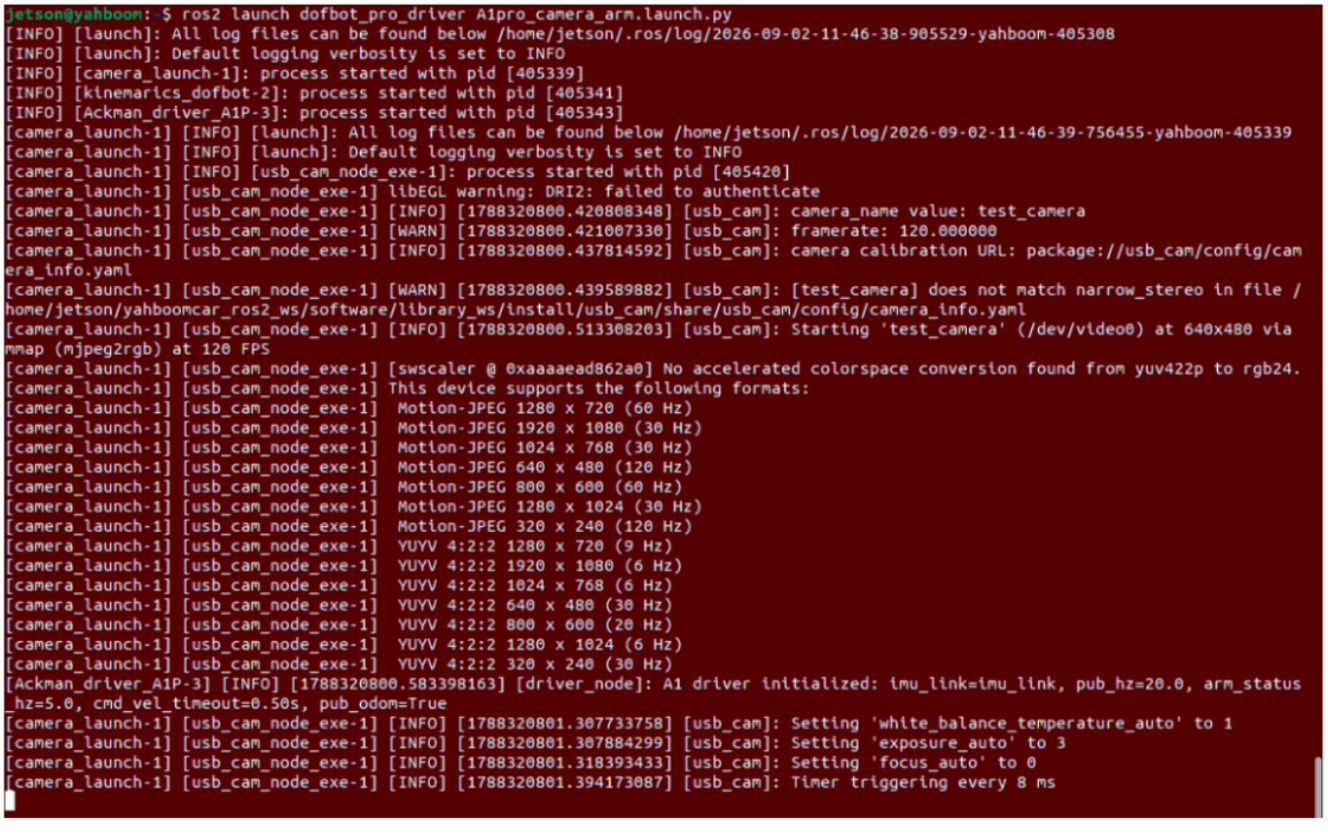
2. Preparation and Running

This case requires opening 3 terminals to start the system sequentially:

2.1 Start Camera Driver and Kinematics Solver Environment

Run in **Terminal 1**:

```
ros2 launch dofbot_pro_driver A1pro_camera_arm.launch.py
```



2.2 Start Robotic Arm Desktop Grasping Action Execution Node

Run in **Terminal 2**:

```
# Depth camera version (Nuwa)
ros2 run dofbot_pro_driver grasp_desktop

# Or USB camera version
ros2 run dofbot_pro_usb grasp_usb
```

```
jetson@yahboom:~$ ros2 run dofbot_pro_usb grasp_usb
{'x_offset': 0.009368814795573138, 'y_offset': 0.007718452024610358, 'z_offset': 0.0}
--------------------------
x_offset:  0.009368814795573138
y_offset:  0.007718452024610358
z_offset:  0.0
--------------------
init done
self.CurEndPose:  [0.0938259179959246, 0.0002628454756523636, 0.12303978979136244, -4.489092034631593e-06, 1.3962632447969856, -2.82
891452582875 86e-05]
array('h', [90, 100, 0, 0, 90, 90])
self.CurEndPose:  [0.0938259179959246, 0.0002628454756523636, 0.12303978979136244, -4.489092034631593e-06, 1.3962632447969856, -2.82
891452582875 86e-05]
```

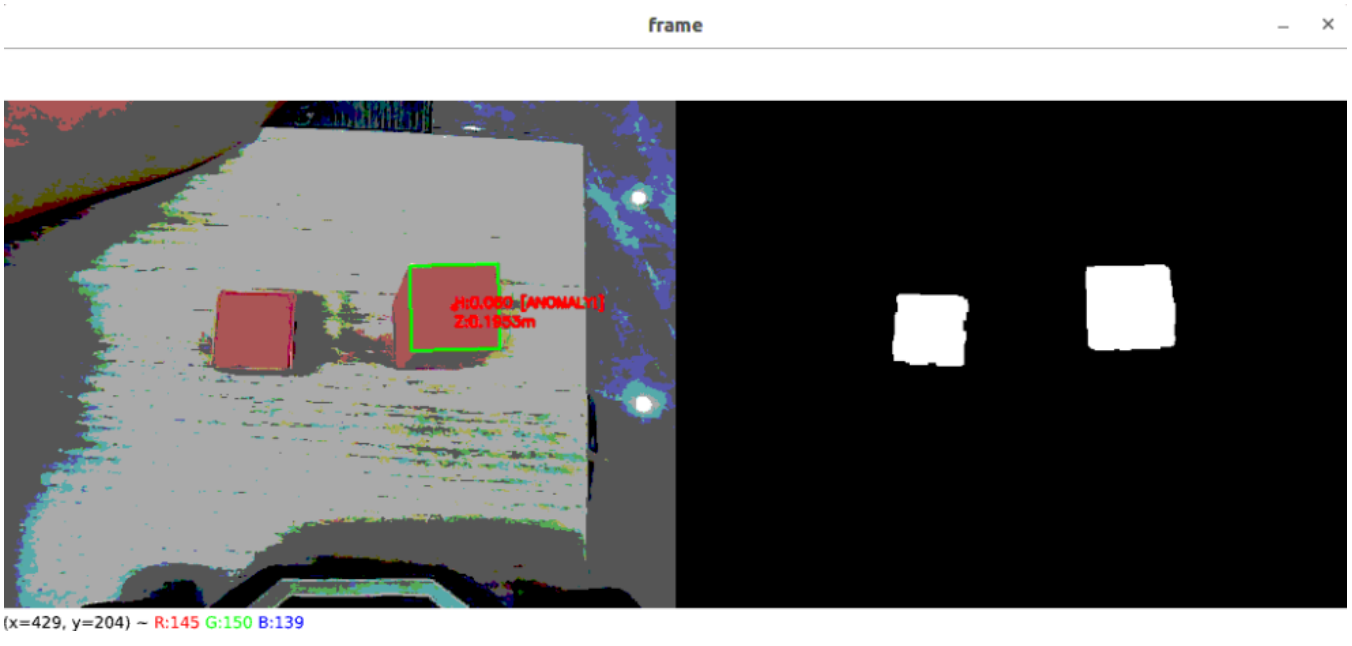
2.3 Start Color Block Height Sorting Detection Node

Run in **Terminal 3**:

```
# Depth camera version (Nuwa)
ros2 run dofbot_pro_driver color_list

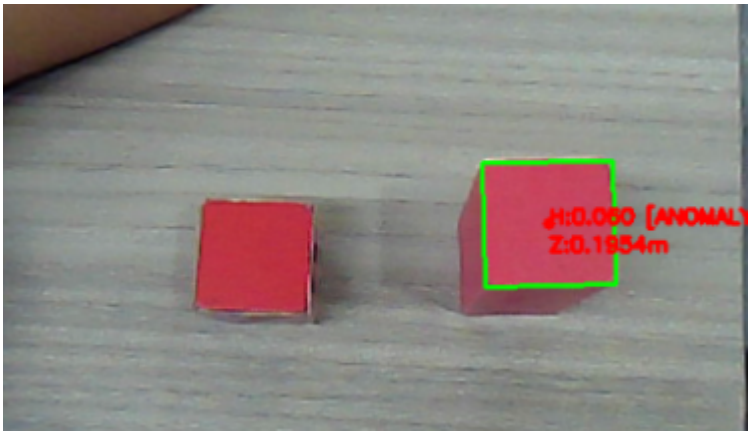
# Or USB camera version
ros2 run dofbot_pro_usb color_usb_height
```

```
jetson@yahboom:~$ ros2 run dofbot_pro_usb color_usb_height
------------------------
[INFO] [1788335482.364980844] [ColorRecognize_node]: object_size: 0.03 m, depth_scale:
0.936504
[INFO] [1788335482.381127405] [ColorRecognize_node]: Color sorting node (dynamic depth
edition) initialized
```



3. Operation Demo and Interaction Flow

1. **Material Placement Preparation**:
    - Prepare normal single-layer blocks (height approximately 30mm) and **double-layer stacked height-anomalous blocks** (height approximately 60mm, stacking two same or different colored blocks together), place them together within the robotic arm field of view;
2. **Select Target Color**:
    - With the image window active, press the **C key** to enter color selection mode;
    - Use the mouse to drag and draw a rectangle covering the target block. The program will automatically extract HSV thresholds from the box and persist them to `custom_colorHSV.text`;
3. **Real-time Height Recognition and Status Annotation**:
    - **USB Monocular Camera**: Single-layer normal items show green `OK` (`H:0.030 [OK]`); double-layer stacked over-standard items show red `ANOMALY!` (`H:0.060 [ANOMALY!]`);
    - **3D Depth Camera**: Directly annotate world frame physical height values next to blocks (e.g., `height: 6.2 cm`);



4. **Trigger Height Anomaly Sorting Grasping**:
    - Press the **spacebar (`Space`)** on the keyboard;
    - The system automatically locks the height-anomalous block (double-layer stacked target), chassis coordinates to fine-tune alignment, robotic arm descends to grasp the top stacked anomalous block and places it in the specified bin;
    - If **all blocks in the current field of view are normal single-layer blocks (no stacking)**, pressing spacebar will not trigger grasping, and the system remains on standby.

4. Core Source Code Analysis

The height sorting system is completed by "color height detection decision" and "IK action execution" two nodes working in coordination:

Camera Type	Color Block Height Detection Decision Node	Grasping Action Execution Node
USB Monocular Camera	`dofbot_pro_usb/color_usb_height.py`	`dofbot_pro_usb/grasp_usb.py`
3D Depth Camera	`dofbot_pro_driver/color_list.py`	`dofbot_pro_driver/grasp_desktop.py`

Unlike regular color sorting, the core of this lesson is: **introducing height filtering mechanism, only triggering grasping when block height exceeds threshold (judged as double-layer stacked anomaly)**.

The main technical implementation comparison between the two:

Comparison Dimension	USB Monocular Camera Solution	3D Depth Camera Solution (Nuwa-HP60C)
Height Calculation Principle	Pinhole geometry calculates depth, derive height by subtracting measured depth from camera calibration reference distance	Physical depth matrix directly indexes block centroid distance, combine with forward kinematics to solve world frame absolute Z-axis height
Anomalous Height Threshold	Height `height_cm > 0.05m` (5cm) judged as anomaly	Height `compute_height_val > 4.0cm` judged as anomaly
Spatial Compensation Dimension	Superimpose X, Y 2D plane calibration compensation	Synchronously superimpose X, Y, Z 3D omnidirectional calibration compensation
Gripper Surface Mechanism	Extract bounding box corner vectors, adjust angle through `compute_joint5`	Extract contour vertex vectors, adjust angle through `compute_joint5`

4.1 USB Monocular Camera Core Source Code Analysis (`color_usb_height.py`)

1. Pinhole Model and Relative Table Height Derivation

The monocular camera calculates ranging depth `cam_z` from the enclosing bounding box pixel size, then derives the block absolute height using the calibrated initial camera height from ground (`0.255m`):

```
# color_usb_height.py —— Relative height derivation
cam_coords = self.estimate_object_camera_coords(
    self.cx, self.cy, self.max_box, fx, fy, cx_cam, cy_cam
)
cam_z = float(cam_coords[2])

# 0.255m is the calibrated reference depth from camera optical center to table at initial pose
# Single-layer block (3cm): depth approximately 0.225m -> height_cm ≈ 0.03m
# Double-layer stacked (6cm): depth approximately 0.195m -> height_cm ≈ 0.06m
height_cm = 0.255 - cam_z
is_anomaly = height_cm > 0.05  # Higher than 5cm judged as double-layer stacked anomaly
```

2. Spacebar Trigger Interception Mechanism

After pressing spacebar, the program strictly checks whether the current block is marked as anomaly, preventing mis-grasping of single-layer normal items:

```
# color_usb_height.py —— Anomaly condition interception
if key == 32:  # Press spacebar
    if self._cur_anomaly and self._cur_cam_coords is not None:
        # Only when height exceeds standard, activate grasping publish flag
        self.pub_pos_flag = True
    else:
        self.get_logger().info('No height-anomaly block found (height_cm <= 0.05), ignoring
trigger.')
```

4.2 3D Depth Camera Core Source Code Analysis (`color_list.py`)

1. Physical Depth Direct Index and World Frame Absolute Height Computation

The depth camera synchronizes images through `ApproximateTimeSynchronizer`, extracts the true physical depth at the block centroid position, and through forward kinematics service and hand-eye calibration matrix, derives the absolute Z-axis height in world coordinates:

```
# color_list.py —— World coordinate absolute physical height computation
if 0 <= cx_int < 640 and 0 <= cy_int < 480:
    dist = depth_image_info[cy_int, cx_int] / 1000.0  # Extract true physical depth (meters)
    pose = self.compute_heigh(cx_int, cy_int, dist)

    # pose[2] is the world coordinate height of the block relative to robotic arm base
    compute_height_val = round(pose[2], 2) * 100  # Convert to centimeters

    # Height greater than 4.0cm and spacebar trigger received, lock anomaly target
    if compute_height_val > 4.0 and self.pub_pos_flag and self.compute_height:
        self.c_x = cx_int
        self.c_y = cy_int
        self.detect_flag = True
```

2. Chassis Alignment and Gripper Rotation Angle Adaptive

After detecting anomalous block, chassis smoothly fine-tunes to 205mm grasping point, while solving joint 5 deflection angle through vertex vectors, enabling the mechanical gripper to close parallel:

```
# color_list.py —— Adaptive rotation and target pose publishing
vx = self.corners[0][0][0] - self.corners[1][0][0]
vy = self.corners[0][0][1] - self.corners[1][0][1]
target_joint5 = compute_joint5(vx, vy)
self.joint5.data = int(target_joint5)

pos = AprilTagInfo()
pos.id = self.target_color  # Carry color ID for subsequent bin sorting
pos.x = float(self.c_x)
```

```
pos.y = float(self.c_y)
pos.z = float(cz)

# Publish message, trigger grasp_desktop node to execute IK grasping
self.pos_info_pub.publish(pos)
self.TargetJoint5_pub.publish(self.joint5)
```

Through this layered and condition interception design, whether using USB monocular camera or 3D depth camera, the robot can sensitively identify blocks with stacking height exceeding standard and execute precise grasping and classified culling.

4.3 Grasping Offset Tuning (Both Camera Models)

If position deviation occurs during actual grasping (such as missing, knocking over objects, offset left/right, or hitting the table), you can directly modify the spatial offset configuration file for the corresponding camera:

1. Where to Modify

Configuration files are located in the `~/yahboomcar_ros2_ws/dofbot_pro_ws/src/dofbot_pro_driver/config/` directory. Select based on camera type:

- **3D Depth Camera (Nuwa-HP60C)**: `offset_value.yaml`
- **USB Camera**: `usb_offset_value.yaml`

2. How to Modify

The file contains offset parameters in X, Y, Z directions (unit: meters, recommended adjustment per tweak is `0.005`~`0.01`, i.e., 5~10mm):

- `x_offset` **(forward/backward direction, positive toward vehicle front)**:
  - Gripper extends too far and knocks over object **decrease value** (adjust smaller or set negative)
  - Gripper doesn't extend far enough and misses **increase value** (adjust larger)
- `y_offset` **(left/right direction, positive toward vehicle left)**:
  - Gripper deviates right **increase value** (correct left)
  - Gripper deviates left **decrease value** (correct right)
- `z_offset` **(up/down height, positive toward ceiling)**:
  - Gripper too high in air and can't grasp **decrease value** (probe deeper)
  - Gripper hits or scrapes table **increase value** (raise higher)